

DrFresh

Ouzas Vasileios Alino Papagiannis

May 12, 2026

ECE490 – IoT System Architecture Proposal

University of Thessaly



Contents

1	System Overview	2
2	Hardware Architecture	2
3	Data Flow & Communication	2
4	Data Modeling	4
5	Data Storage Strategy	5
6	Processing & System Logic	6
7	Reliability Considerations	8
8	Data Visualization	8
9	Design Justification	10
10	Implementation Reality Check	11

1 System Overview

DrFresh is a smart liquid refreshment dispenser designed to automate and monitor the serving of beverages from two separate tanks. The system addresses the need for controlled, trackable, and user-friendly liquid dispensing in environments such as offices or small businesses, where both convenience and usage monitoring are important. The system also implements an alerting system, letting the "owners" know when any of the tanks needs a refill, minimizing the time for which any tank stays empty.

The system is built around a Raspberry Pi acting as the central controller. It manages two water pumps through a dual-channel relay module, handles communication via the MQTT protocol, and maintains operational state in a local SQLite database. Dispensing events, alerts, tank volume snapshots, and computed analytics are recorded in InfluxDB and visualized through a Grafana dashboard. The system supports two dispensing modes: an automatic mode that dispenses a fixed (customizable through code) default volume, and a manual mode where the user controls the duration of dispensing by sending continuous heartbeat messages. Safety mechanisms ensure pumps are never left running unintentionally, even under network or user error conditions.

2 Hardware Architecture

The hardware architecture revolves around the Raspberry Pi as the main control and processing unit. It combines GPIO hardware control, networking and application logic in a single device, which simplifies the overall system design.

A dual-channel relay module is connected to two GPIO pins on the Raspberry Pi (BCM pins 17 and 27 for pumps A and B respectively). The relay module acts as a safe electrical switch, allowing the low-voltage GPIO signals from the Pi to control the 6–12 V water pumps without direct electrical coupling. Both pumps are capable of a maximum flow rate of 3 L/min and are connected to their respective liquid tanks via 7×10 mm silicone tubing. A 12 V external power supply powers the pumps.

All GPIO pins are initialized in the OFF state (HIGH) at startup, ensuring that pumps cannot activate unintentionally following a power interruption or system restart. This forms a complete cyber-physical system where the Raspberry Pi coordinates estimation, actuation, communication, and storage.

3 Data Flow & Communication

The system uses two distinct communication layers: an HTTP layer between the browser-based GUI and the Raspberry Pi, and an MQTT layer between the Raspberry Pi and the broader IoT infrastructure. These two layers are fully independent — the GUI never

communicates with the MQTT broker directly.

GUI to Raspberry Pi — HTTP

The GUI is a single-page application served by a Flask web server running on the Raspberry Pi at port 5000. All user interactions reach the backend through three HTTP endpoints:

- `POST /api/dispense` — accepts a JSON payload with `tank` ("A" or "B") and `type` ("auto" or "manual"). The server validates the input and calls `handle_command()` directly in the same process.
- `POST /api/refill` — accepts a JSON payload with `tank` and optionally `volume`. If no volume is specified, the tank is assumed to be filled to the maximum capacity of 1.5L. The server calls `handle_refill()` directly.
- `GET /api/stream` — a Server-Sent Events (SSE) endpoint that pushes real-time state updates to the browser every 2 seconds. Each event carries the current estimated volume of both tanks, read directly from the SQLite `tanks` table. If the state has not changed since the last push, a heartbeat comment is sent to keep the connection alive.

The Flask server and the control script run in the same Python process: the `script.main()` function is launched as a daemon thread when the Flask server starts, and the HTTP endpoint handlers call the control functions directly with no inter-process communication.

Manual Dispensing — Hold to Dispense

In manual mode the user holds the Dispense button in the GUI. While held, the browser sends a `POST /api/dispense` request with `"type": "manual"` every 250 ms via `setInterval`. Each request resets the watchdog timer on the Pi. When the user releases the button the interval is cleared; the Pi detects the absence of a pulse within 0.75 seconds and closes the pump automatically. The GUI estimates the dispensed volume client-side as elapsed time multiplied by the flow rate and briefly displays it to the user.

Raspberry Pi to IoT Infrastructure — MQTT

The Raspberry Pi connects as an MQTT client to a shared remote broker at `194.177.207.38:1883` using username and password authentication. All topics are namespaced under `iot/team1/drfresh/`. The client is assigned a unique ID generated at startup from a UUID prefix to prevent session conflicts on reconnection.

The Pi subscribes to two topics for receiving commands from external clients:

- `iot/team1/drfrsh/command` — external clients may publish dispense commands here; the Pi processes them identically to HTTP-originated commands.
- `iot/team1/drfrsh/refill` — external clients may publish refill events here; the Pi also publishes a confirmation payload back on this topic after processing a refill.

The Pi publishes to one topic:

- `iot/team1/drfrsh/alert` — published whenever a low-tank, empty-tank, watchdog closure, or tank-ran-empty condition is detected.

The MQTT client is configured with automatic reconnection using exponential backoff (minimum 1 second, maximum 30 seconds). Because all GUI-facing logic goes through HTTP and local SQLite, a broker outage has no effect on normal dispensing operation.

4 Data Modeling

The system's data model follows the NGSI-LD standard. The DrFresh dispenser is represented as a single entity in the Orion-LD context broker, identified as `urn:ngsi-ld:DrinkDispenser:team1`. The entity is created on startup and updated after every refill event via HTTP PATCH to the broker's `/ngsi-ld/v1/entities/{id}/attrs` endpoint.

Tank levels are expressed as fill percentages, computed as the estimated volume divided by the maximum tank capacity of 1.5 L and rounded to one decimal place. The `status` field reflects the most recent lifecycle event: it is set to `initialized` on startup (both at entity creation and on a 409 Conflict fallback), and updated to `refilled` after each refill event. No other operations trigger an Orion-LD update — dispense events update SQLite and InfluxDB only.

All HTTP requests to Orion-LD include a `Link` header referencing the NGSI-LD core context and use `Content-Type: application/json`. All calls are wrapped in `try/except` with a 3-second timeout so a broker outage never interrupts dispensing logic.

The JSON example below shows the entity structure as posted on startup.

Listing 1: NGSI-LD representation of the DrFresh dispenser entity

```
1 {
2   "id": "urn:ngsi-ld:DrinkDispenser:team1",
3   "type": "DrinkDispenser",
4
5   "tankLevelA": {
6     "type": "Property",
7     "value": 100.0,
8     "unitCode": "P1"
```

```
9   },
10
11   "tankLevelB": {
12     "type": "Property",
13     "value": 100.0,
14     "unitCode": "P1"
15   },
16
17   "status": {
18     "type": "Property",
19     "value": "initialized"
20   },
21
22   "@context": [
23     "https://uri.etsi.org/ngsi-ld/v1/ngsi-ld-core-context.jsonld"
24   ]
25 }
```

The `unitCode` value `P1` denotes percentage in the UN/CEFACT unit code standard. The inclusion of the standard `@context` enables semantic interoperability with other NGSI-LD-compliant systems and IoT platforms.

5 Data Storage Strategy

The system generates event-based data, where each dispensing action, refill, or alert produces a timestamped record. The main data types produced are:

- **Drink Type:** identifies whether Tank A or Tank B was selected.
- **Dispense Type:** indicates whether the operation was automatic or manual.
- **Amount Dispensed:** the volume in litres served during the operation.
- **Status:** confirms whether the dispensing completed successfully or failed.
- **Timestamp:** records the exact date and time of the event.
- **Alert Type:** describes any warning or fault condition (e.g. low tank, empty tank, watchdog closure).
- **Tank Volume:** the estimated remaining liquid in each container, updated after every dispense or refill.

Two storage solutions were evaluated: SQLite and InfluxDB. The final system uses both

in a complementary hybrid strategy.

SQLite is a lightweight relational database that stores all data in a single local file and requires no separate server process. In DrFresh, SQLite is used exclusively to store the current estimated volume of each tank in a **tanks** table. When a dispense command arrives, the Pi queries this table synchronously to verify whether sufficient liquid is available before activating the pump, and updates it immediately after. This read/write is fast, local, and requires no network dependency. The Flask SSE stream also reads from this table every 2 seconds to push live volume updates to the GUI.

InfluxDB is a time-series database optimized for storing and querying timestamped measurements, and integrates natively with Grafana. In DrFresh, InfluxDB stores three measurement types: **dispense_event** (tank, type, status, amount), **tank_volume** (volume snapshot per tank), and **alert** (alert message per tank). After each event the Pi also queries these measurements to compute and write a derived **analytics** point per tank, containing: total volume dispensed, dispense count, average volume per dispense, auto count, manual count, auto/manual ratio, refill count, low-tank alert count, empty-tank attempt count, and tank-ran-empty count.

This hybrid approach separates concerns cleanly: SQLite serves the real-time control logic and the GUI volume stream with minimal overhead, while InfluxDB serves the analytics and visualization layer. Grafana reads exclusively from InfluxDB, meaning no direct access to the SQLite file is needed from outside the control process.

6 Processing & System Logic

The system follows an event-driven architecture, where user interactions through the web-based GUI generate events that trigger specific system actions. For example, selecting a tank, initiating a dispense operation, or requesting a refill produces an event that is sent to the Raspberry Pi controller. Before performing any physical actuation, the controller evaluates a set of rule-based conditions, such as tank availability, minimum volume thresholds, and operating mode constraints. Only when all validation rules are satisfied does the system activate the corresponding pump or process the requested action.

The system supports two dispensing modes. In **automatic mode**, a fixed default volume of 0.3 L is dispensed (This volume is also easily customizable through the python code). When a command is received, the Pi queries SQLite to confirm that the selected tank has at least 0.3 L remaining. If the check passes, GPIO pin 17 (Tank A) or pin 27 (Tank B) is driven LOW to activate the corresponding relay and start the pump. The pump runs for the time required to dispense 0.3 L at the configured flow rate of 0.03 L/s (10 seconds), after which the pin is driven HIGH to stop the pump. The volume in SQLite is then decremented, the dispense event and a tank volume snapshot are written to InfluxDB,

and the Pi computes and stores updated analytics (including running totals and depletion trends) in InfluxDB. The alerting mechanism is integrated into the system's decision layer and operates based on tank volume thresholds. When the estimated liquid volume falls below a predefined limit (0.3 L), in addition to visual notifications on the GUI, the alerting mechanism can be extended to support remote notifications via e-mail with the help of the Gmail SMTP (smtplib). This allows operators to be informed of low tank levels even when they are not actively monitoring the application, enabling timely refill operations and reducing system downtime. Apart from the mailing system, the integrated MQTT protocol publishes to the `iot/team1/drfrsh/refill` and `iot/team1/drfrsh/alert` topics each time there is a refill and a low tank level event respectfully, allowing external applications and monitoring services to subscribe and receive real-time information

If the remaining volume after dispensing falls below the low-tank threshold of 0.1 L, an alert is published to `iot/team1/drfrsh/alert` and also recorded in InfluxDB.

In **manual mode**, the user initiates dispensing by sending a manual command and keeps the pump running by continuously sending heartbeat messages to `drfrsh/command`. The system implements a watchdog mechanism: if no heartbeat is received within a 0.75-second window, the pump is automatically shut off. This protects against scenarios where the user loses connectivity or simply walks away without explicitly stopping the dispense. Additionally, a timeout is calculated at the start of each manual session as `current_volume/FLOW_RATE`. If this timeout elapses, it means the tank would be empty and the pump is closed automatically, preventing the pump from running dry.

Typical scenario: a user approaches the dispenser and connects to 10.24.4.20:5000. They select Tank A, they choose automatic dispensing and they press the "Auto dispense 0.3L" button. The Pi checks SQLite and finds Tank A has 0.45 L remaining, which is above the 0.3 L minimum required. The pump activates for 10 seconds and dispenses 0.3 L. The Pi updates SQLite to 0.15 L, writes the dispense event and a tank volume snapshot to InfluxDB, updates the running analytics stored in InfluxDB (such as total volume dispensed from Tank A and the current depletion trend). Since 0.15 L is below the 0.3 L default amount, an email alert is sent to the "owner", letting them know that they need to refill the tank. Moreover, the PI publishes an alert over MQTT at the `iot/team1/drfrsh/alert` topic suggesting that the volume has dropped below the default amount.

Refill events are handled through the `iot/team1/drfrsh/refill` topic. The Pi adds this to the current SQLite volume, capped at the maximum tank capacity of 1.5 L, updates SQLite, writes a refill event and a tank volume snapshot to InfluxDB, updates the running analytics in InfluxDB accordingly, and publishes the new volume to `drfrsh/volume`.

7 Reliability Considerations

The DrFresh system is designed to maintain safe and reliable operation under common failure conditions including communication loss, invalid input, and hardware faults.

If the MQTT broker or network connection becomes temporarily unavailable, the Raspberry Pi continues executing local control logic. Commands already received can still be processed. The MQTT client is configured with automatic reconnection using exponential backoff (1 to 30 seconds), so the connection is restored without manual intervention once the broker is reachable again.

The manual dispensing watchdog is a key reliability mechanism. Because the pump remains active only as long as the client keeps sending heartbeat messages within 0.75-second intervals, any client disconnection — whether due to a crash, network drop, or user inaction — causes the pump to shut off automatically. This prevents accidental over-dispensing and protects both the hardware and the liquid supply.

Each manual session also carries a calculated timeout based on the remaining tank volume divided by the flow rate. This acts as a hard upper bound on how long the pump can run, independently of the watchdog. If the tank would run empty, the pump closes and an alert is recorded.

All GPIO pins are initialized HIGH (pump off) during startup, ensuring pumps cannot activate unintentionally after a power interruption or reboot. Invalid commands received via MQTT are rejected through software validation before any GPIO action is taken. Impossible refill values are also handled gracefully, with the volume capped at the maximum tank capacity.

SQLite is stored locally on the Raspberry Pi, making tank state available even during network outages. InfluxDB, if hosted locally, similarly retains all historical data independently of external connectivity. The separation of operational state (SQLite) from analytics storage (InfluxDB) means a failure in one does not affect the other: if InfluxDB is temporarily unavailable, the control logic continues uninterrupted.

8 Data Visualization

The DrFresh system provides two complementary visualization interfaces: a browser-based GUI for real-time operator interaction, and a Grafana dashboard for historical analysis and system monitoring.

Browser GUI

The GUI is a single-page HTML application served by Flask directly from the Raspberry Pi. It connects to the backend via a Server-Sent Events stream (`/api/stream`) that pushes volume updates every 2 seconds without requiring the user to refresh.

The interface displays:

- Two animated bottle graphics whose fill height updates in real time to reflect the estimated volume of Tank A and Tank B. The fill color transitions from blue (above 70% capacity) to yellow (above 30%) to red (below 30%), giving an immediate visual indication of tank health.
- The current estimated volume in litres displayed below each bottle.
- Tank selection and mode selection (Auto / Manual) buttons, and a main Dispense button whose label and enabled state update dynamically based on the current selection and tank volumes.
- An alert banner that appears automatically when a tank drops below 0.3 L, indicating which tank is low.
- Refill buttons for Tank A and Tank B that appear when a low-tank condition is detected. Each refill button sends a `POST /api/refill` request that resets the selected tank to full capacity (1.5 L).

Grafana Dashboard

The Grafana dashboard provides long-term monitoring and is organized into three panel groups, all reading from InfluxDB via Flux queries.

The **main overview panel** contains a pie chart of total dispense count split by tank, a dual time-series graph of tank levels over the past 30 days, a time-series graph of refill events, and stat panels showing the current live estimated volume of each tank.

The **alerts and history panel** contains a table of all alert events with timestamps and tank identifiers, a dispense history time-series graph, stat panels showing refill counts per tank (10 for Tank A, 8 for Tank B as of the latest snapshot), and stat panels showing total cumulative volume dispensed (8.76 L for Tank A, 12.7 L for Tank B).

The **analytics panel** contains stat panels for the auto/manual dispense ratio per tank (25% auto for Tank A, 21.3% for Tank B) and the average volume per manual session (125 mL for Tank A, 120 mL for Tank B).

Two visualization tools were considered: **Grafana** and **ThingsBoard**. Grafana was

selected for its native InfluxDB and Flux query integration, its range of panel types (time-series, pie charts, stat panels, tables), and its ability to self-host on the Raspberry Pi with minimal resource usage. ThingsBoard, while a more complete IoT platform with built-in device management and rule chains, introduces significantly higher complexity and resource requirements that are unnecessary for a two-tank single-device system.

9 Design Justification

The system architecture was selected to balance simplicity, reliability, and ease of implementation while satisfying all project requirements.

The Raspberry Pi was chosen as the central controller because it combines GPIO hardware control with full networking capabilities and Python software support, allowing a single device to manage communication, processing, storage, and actuation without additional microcontrollers.

The decision to serve the GUI via Flask and HTTP rather than a WebSocket MQTT client significantly simplifies the browser-side architecture. The GUI only needs standard HTTP and SSE support; it requires no broker credentials and no WebSocket library. The Flask server and control script run in the same Python process, so HTTP requests invoke the control logic directly with no inter-process overhead. MQTT is retained solely for integration with the shared broker and the Orion-LD context broker, not for GUI communication.

The hybrid storage strategy — SQLite for operational state, InfluxDB for event history and analytics — reflects a deliberate separation of concerns. SQLite provides synchronous, low-latency reads and writes for both the control logic and the GUI's SSE stream without any network dependency. InfluxDB provides a purpose-built time-series store for raw events, volume snapshots, alerts, and computed analytics, with native Grafana integration.

The buzzer provides a local, network-independent alert that fires immediately on the device without depending on the broker or dashboard. The email alert complements this by reaching the operator remotely and asynchronously. Both are non-blocking: the buzzer runs on a daemon thread and the email call is wrapped in try/except so neither can interrupt the control loop.

Grafana was chosen for its native InfluxDB integration, lightweight self-hosting, and panel variety. Overall, the chosen components satisfy all project requirements while keeping implementation complexity appropriately low for a two-tank prototype.

10 Implementation Reality Check

One practical challenge of the system is that tank levels are estimated rather than directly measured. Since no dedicated liquid level sensors are used, the remaining volume is calculated by decrementing from a known starting value based on cumulative dispensing time and the configured flow rate of 0.03 L/s. This method is cost-effective but can accumulate small errors over time, particularly if the actual pump flow rate varies due to pressure differences or partial blockages. The same principal applies to the tanks. Since they are purely virtual, there may be some minor inconsistencies regarding their actual volume.

The system currently relies on a local Raspberry Pi for control and decision-making, while MQTT is used only for event notifications rather than full telemetry streaming. This choice reduces implementation complexity but limits the amount of real-time operational data available to external subscribers.

Furthermore, the web interface is intended for use within a local network and does not currently include authentication, user management, or encrypted communication. While these features are not essential for a laboratory prototype, they would be necessary in a production deployment.

Despite these limitations, the implemented prototype successfully demonstrates the core concepts of IoT-based monitoring, event-driven control, rule-based decision logic, remote visualization, and MQTT-enabled event notification.

The layered IoT system architecture of DrFresh is illustrated in Figure 1.

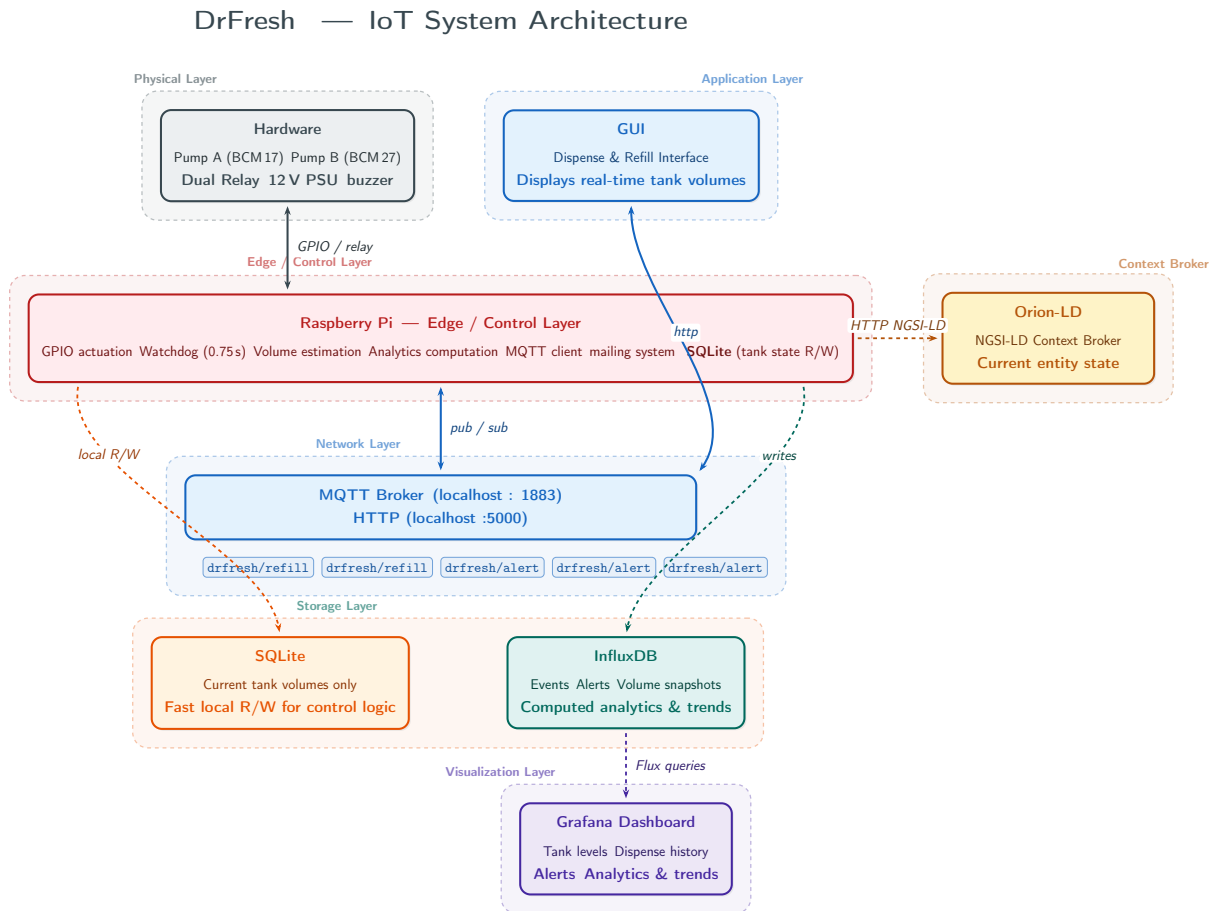


Figure 1: Solid double arrows = bidirectional data flow. Dashed arrows = direct local writes bypassing the broker.

Figure 1: System architecture of the DrFresh smart drink dispenser. Solid arrows indicate bidirectional or directed data flow; dashed arrows indicate direct local writes.